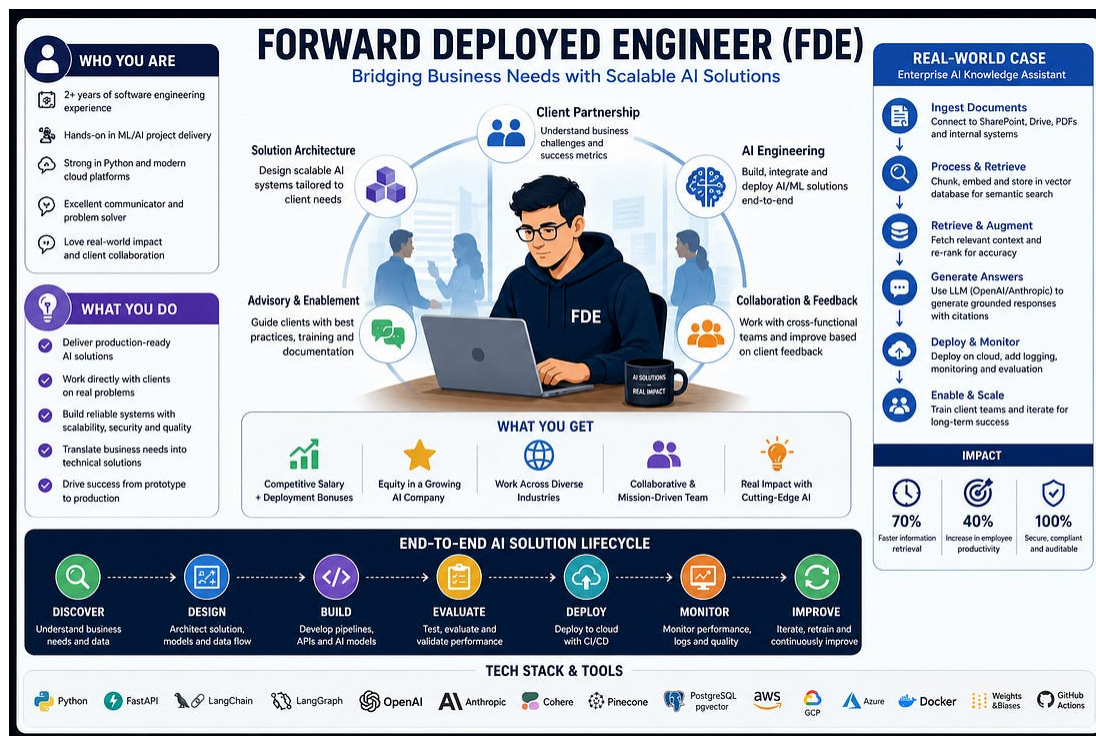


Forward Deployed Engineer (FDE) Mock Interview Q&A



AI ENGINEERING INSIDER

MAY 19, 2026



1. Tell me about yourself.

"I'm an AI-focused software engineer with experience building automation systems, AI integrations, and production-oriented workflows across backend engineering, evaluation, and deployment. My background combines software engineering, AI systems, and customer-facing collaboration.

This Substack is reader-supported. To receive new posts and support my work,

consider becoming a free or paid subscriber.

✓ **Subscribed**

Recently, I've been focused on building agentic AI and RAG systems using Python, FastAPI, LangChain, LangGraph, vector databases, and OpenAI APIs. I enjoy translating ambiguous business problems into deployable technical systems.

What interests me about the Forward Deployed Engineer role is the combination of hands-on engineering and direct client interaction. I like working closely with stakeholders, rapidly prototyping solutions, iterating based on feedback, and helping clients successfully operationalize AI systems in production.”

Client-Facing & Consulting Questions

2. A client says: “We want AI.” How would you handle the conversation?

“I would avoid jumping directly into tools or models first.

I would start by understanding:

- ↳ What business problem they are trying to solve
- ↳ Current workflow pain points
- ↳ Existing systems and infrastructure
- ↳ Success metrics
- ↳ Constraints like privacy, latency, or compliance

Then I would translate those findings into technical opportunities.

For example:

- ↳ If they need knowledge retrieval → RAG system
- ↳ If they need automation → AI workflow orchestration
- ↳ If they need prediction → ML model pipeline
- ↳ If they need copilots → LLM integration

I would rapidly prototype a thin vertical slice first rather than overengineering the solution.”

3. How do you explain RAG to non-technical stakeholders?

“I explain Retrieval-Augmented Generation as giving the AI access to company knowledge before answering.

Without RAG:

- ↳ The model answers from general training data

With RAG:

- ↳ The system first searches relevant internal documents
- ↳ Retrieves the best matching content
- ↳ Injects it into the prompt
- ↳ Then generates grounded answers

This reduces hallucination and allows the assistant to answer using company-specific information.”

4. How would you handle a difficult client demanding unrealistic timelines?

“I would focus on scope control and phased delivery.

Instead of saying 'no,' I would:

- ↳ Break work into milestones
- ↳ Prioritize highest-impact features
- ↳ Deliver a working prototype quickly
- ↳ Clearly communicate risks and tradeoffs

For example:

- ↳ Week 1: document ingestion + retrieval
- ↳ Week 2: chatbot interface
- ↳ Week 3: evaluation and monitoring

That keeps momentum while maintaining realistic engineering quality.”

AI Engineering Questions

5. Explain a production RAG architecture.

“A production RAG architecture usually includes:

- ↳ Document ingestion pipeline
- ↳ Chunking strategy
- ↳ Embedding generation
- ↳ Vector database
- ↳ Retrieval layer
- ↳ Re-ranking layer
- ↳ LLM orchestration
- ↳ Evaluation system
- ↳ Observability and tracing

Typical flow:

User Query

→ Embedding

- Vector Search
- Retrieve Top-K Chunks
- Re-rank Results
- Inject Context into Prompt
- LLM Response
- Logging + Evaluation

I also add:

- ↳ Citations
- ↳ Confidence scoring
- ↳ Fallback handling
- ↳ Human escalation paths”

6. What vector databases have you used?

“I’ve worked with systems like:

- ↳ Chroma
- ↳ PGVector
- ↳ Pinecone
- ↳ Qdrant

The choice depends on:

- ↳ Scale
- ↳ latency requirements
- ↳ operational complexity
- ↳ cloud environment
- ↳ filtering requirements

For smaller deployments, PGVector is often sufficient and operationally simpler because it stays inside PostgreSQL.”

7. How do you reduce hallucinations in LLM systems?

“I use multiple layers:

- ↳ Retrieval grounding
- ↳ Strict prompt instructions
- ↳ Re-ranking retrieved chunks
- ↳ Citation enforcement
- ↳ Structured output validation
- ↳ Confidence thresholds
- ↳ Evaluation pipelines
- ↳ Human escalation workflows

I also avoid giving the model unnecessary freedom when deterministic outputs are needed.”

8. Explain prompt engineering in production systems.

“In production systems, prompt engineering is less about clever prompts and more about reliability.

I focus on:

- ↳ Structured templates
- ↳ Guardrails
- ↳ Explicit formatting rules
- ↳ Few-shot examples
- ↳ Role separation
- ↳ Output schemas

For example:

- ↳ Returning strict JSON

- ↳ Forcing citations
 - ↳ Constraining tool usage
 - ↳ Preventing unsupported claims”
-

System Design Questions

9. Design an enterprise internal AI assistant.

“I would design:

Backend

- ↳ FastAPI services
- ↳ LangChain orchestration
- ↳ Authentication layer

Data Layer

- ↳ SharePoint / Google Drive connectors
- ↳ Chunking + embeddings
- ↳ Vector database

Retrieval

↳ Hybrid retrieval:

- BM25
- semantic search
- reranking

LLM Layer

- ↳ OpenAI or Anthropic APIs

Security

- ↳ RBAC access control
- ↳ encrypted storage
- ↳ audit logging

Observability

- ↳ LangSmith or OpenTelemetry tracing
- ↳ evaluation dashboards
- ↳ prompt/version tracking”

10. How would you deploy an AI application?

“I usually containerize applications with Docker and deploy using cloud-native services.

Example stack:

- ↳ FastAPI backend
- ↳ Docker containers
- ↳ AWS ECS or GCP Cloud Run
- ↳ PostgreSQL + pgvector
- ↳ Redis caching
- ↳ CI/CD via GitHub Actions

Deployment pipeline:

- ↳ test
 - ↳ lint
 - ↳ build container
 - ↳ push image
 - ↳ deploy automatically”
-

Full-Stack Engineering Questions

11. How would you build a FastAPI AI service?

"I would structure it into:

- ↳ API routes
- ↳ service layer
- ↳ retrieval layer
- ↳ model orchestration layer
- ↳ logging/tracing layer

Key considerations:

- ↳ async processing
- ↳ retry logic
- ↳ rate limiting
- ↳ structured logging
- ↳ caching
- ↳ secret management

I would also separate prompt templates and configuration from business logic."

12. Explain CI/CD for AI systems.

"AI CI/CD includes traditional software validation plus AI-specific checks.

Pipeline:

- ↳ unit tests
- ↳ integration tests
- ↳ prompt regression tests

- ↳ evaluation benchmarks
- ↳ hallucination checks
- ↳ latency monitoring

For example:

- ↳ before deployment, compare answer quality against baseline evaluations.”

Behavioral Questions

13. Tell me about a time you handled ambiguity.

“At one project, requirements were initially unclear because stakeholders only knew they wanted ‘AI automation.’

I organized discovery sessions to identify:

- ↳ repetitive workflows
- ↳ data sources
- ↳ approval bottlenecks
- ↳ operational pain points

From there, I proposed a phased architecture instead of a massive platform rewrite.

That helped us rapidly deliver measurable value while refining requirements iteratively.”

14. Tell me about a technical failure.

“One challenge was retrieval quality degradation in a document assistant.

The root causes were:

- ↳ poor chunking strategy
- ↳ overlapping irrelevant context
- ↳ missing metadata filters

I improved:

- ↳ chunk sizing
- ↳ metadata tagging
- ↳ hybrid retrieval
- ↳ reranking

That significantly improved grounded answer quality.”

15. Why do you want to be a Forward Deployed Engineer?

“I enjoy the intersection of engineering and real-world problem solving.

I like:

- ↳ building production systems
- ↳ interacting directly with users
- ↳ understanding operational workflows
- ↳ rapidly iterating solutions
- ↳ seeing immediate business impact

The FDE role is appealing because it combines technical depth with customer collaboration rather than being isolated from deployment realities.”

Advanced AI Questions

16. Explain LangChain vs LangGraph.

“LangChain is useful for linear orchestration:

- ↳ chains
- ↳ retrieval
- ↳ tool calling

LangGraph is better for:

- ↳ multi-agent workflows
- ↳ stateful execution
- ↳ branching logic
- ↳ retries
- ↳ planner/executor systems

I typically use LangGraph when workflows require loops, memory, verification, or agent coordination.”

17. What is an AI evaluation pipeline?

“An evaluation pipeline measures:

- ↳ correctness
- ↳ groundedness
- ↳ hallucination rate
- ↳ latency
- ↳ retrieval quality
- ↳ tool accuracy

I usually combine:

- ↳ automated benchmark datasets
- ↳ LLM-as-judge

- ↳ human review
- ↳ tracing systems

This helps prevent regressions when prompts or models change.”

18. How do you secure enterprise AI systems?

“I apply:

- ↳ RBAC authorization
- ↳ encrypted storage
- ↳ secure secret management
- ↳ audit logging
- ↳ PII filtering
- ↳ network isolation
- ↳ access-scoped retrieval

I also ensure prompts and logs do not leak sensitive enterprise data.”

Strong Closing Question to Ask Interviewer

“From your experience, what differentiates the strongest Forward Deployed Engineers on your team from engineers who struggle in client-facing deployments?”

This Substack is reader-supported. To receive new posts and support my work, consider becoming a free or paid subscriber.

✓ Subscribed